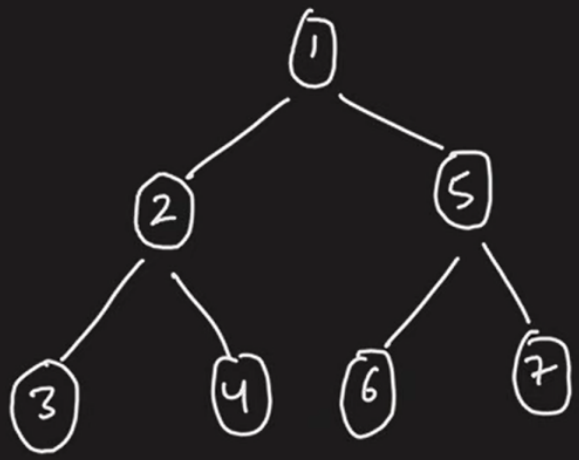
2.20

Preorder / Inorder / Postorder Traversals



Preorder → 1 2 3 4 5 6 7

Inorder → 3 2 4 1 6 5 7

Postorder → 3 4 2 6 7 5 1

(node, num)

Stack

↳ LIFO

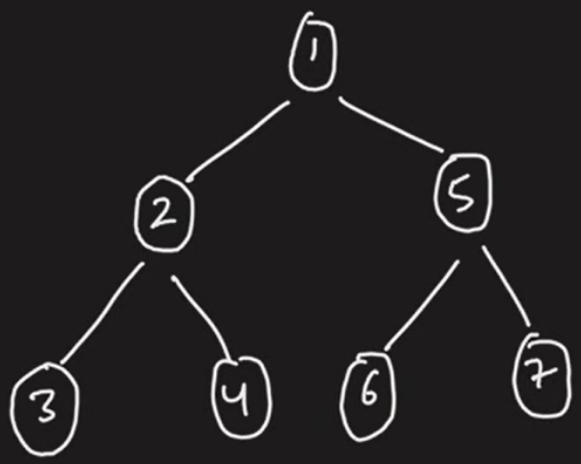
if num = 1
preorder
++
left

if num == 2
inorder
++
right

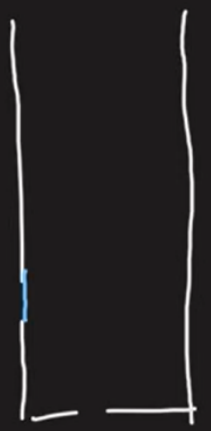
if num == 3
postorder

2.20

Preorder / Inorder / Postorder Traversals



TC → O(3xN)
SC → O(4N)



(node, num)

Stack
↳ LIFO

if num = 1
preorder
++
left

if num == 2
inorder
++
right

if num == 3
postorder

Preorder → 1 2 3 4 5 6 7

Inorder → 3 2 4 1 6 5 7

Postorder → 3 4 2 6 7 5 1

```
 */
class Solution {

public:
    vector<int> preInPostTraversal(TreeNode* root) {
        stack<pair<TreeNode*,int>> st;
        st.push({root, 1});
        vector<int> pre, in, post;
        if(root == NULL) return;
        while(!st.empty()) {
            auto it = st.top();
            st.pop();

            // this is part of pre
            // increment 1 to 2
            // push the left side of the tree
            if(it.second == 1) {
                pre.push_back(it.first->val);
                it.second++;
                st.push(it);

                if(it.first->left != NULL) {
                    st.push({it.first->left, 1});
                }
            }

            // this is a part of in
            // increment 2 to 3
            // push right
            else if(it.second == 2) {
                in.push_back(it.first->val);
                it.second++;
                st.push(it);

                if(it.first->right != NULL) {
                    st.push({it.first->right, 1});
                }
            }
            // don't push it back again
            else {
                post.push_back(it.first->val);
            }
        }
    }
};
```